

Theoretische Informatik

Alphabete, Wörter und Sprachen

Alphabet

Ein Alphabet ist eine endliche, nichtleere Menge von Symbolen. Das Bsp.:

$$\Sigma = \{a, b, c\}, \quad \Sigma_{\text{bool}} = \{0, 1\}, \quad \Sigma_{\text{lat}} = \{a, b, c \cdots, z\}$$

Wörter

Ein Wort über einem Alphabet ist eine endliche Folge von Symbolen aus diesem Alphabet. Das leere Wort wird mit ε bezeichnet.

$$w = (x_1, x_2, \dots, x_n), \quad x_i \in \Sigma, \quad n \in \mathbb{N}$$

Die Länge eines Wortes w ist die Anzahl der Symbole, also $|w| = n$. Die absolute Häufigkeit eines Symbols a in einem Wort w wird mit $|w|_a$ bezeichnet. Zusätzlich gilt:

$$|\varepsilon| = |\varepsilon|_a = 0, \quad a \in \Sigma$$

Spiegelung und Palindrome

Mit w^R wird das Spiegelwort von w bezeichnet, also die Umkehrung der Symbolfolge.

$$w^R = (x_1, x_2, \dots, x_n)^R = (x_n, x_{n-1}, \dots, x_1)$$

Wenn $w = w^R$ gilt, dann ist w ein Palindrom.

Wörter der Länge k

Die Menge aller Wörter der Länge k über einem Alphabet Σ wird mit Σ^k bezeichnet:

$$\Sigma^k = \{w \in \Sigma^* \mid |w| = k\}$$

Unabhängig von Σ gilt stets $\Sigma^0 = \{\varepsilon\}$.

Kleenesche Hülle

Die Menge aller Wörter über einem Alphabet Σ wird mit Σ^* bezeichnet:

$$\Sigma^* = \bigcup_{k \geq 0} \Sigma^k$$

Die Menge aller nichtleeren Wörter über Σ wird mit Σ^+ bezeichnet:

$$\Sigma^+ = \Sigma^* \setminus \{\varepsilon\}$$

Teilwortrelationen

Infix v ist Infix von $w \iff$ Es existieren $x, y \in \Sigma^*$ mit $w = xvy$.

Präfix v ist Präfix von $w \iff$ Es existiert $y \in \Sigma^*$ mit $w = vy$.

Suffix v ist Suffix von $w \iff$ Es existiert $x \in \Sigma^*$ mit $w = xv$.

Man spricht von einem *echten* Infix/Präfix/Suffix, wenn $v \neq w$ gilt.

Konkatination

Die Konkatenation von zwei Wörtern x und y ist die Aneinanderreihung der Symbole von x gefolgt von den Symbolen von y :

$$x \circ y = xy = (x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_m)$$

Die Länge der Konkatenation ist die Summe der Längen der beiden Wörter:

$$|xy| = |x| + |y|$$

Die Konkatenation mit dem leeren Wort ε hat keine Auswirkung auf das Wort:

$$w\varepsilon = \varepsilon w = w$$

Wortpotenzen

Die n -te Potenz eines Wortes x wird für alle $n \in \mathbb{N}$ rekursiv definiert als

$$\begin{aligned} x^0 &:= \varepsilon \\ x^{n+1} &:= x^n x \end{aligned}$$

Die Potenzierung mit der Kleeneschen Hülle ergibt die gleiche Menge:

$$(A^*)^* = A^*$$

Sprache

Eine Teilmenge $L \subseteq \Sigma^*$ von Wörtern heisst Sprache über dem Alphabet Σ . Es gelten die folgenden Eigenschaften:

- $\{\}$ ist die leere Sprache für jedes Alphabet Σ .
- $\{\varepsilon\}$ ist die Sprache, die nur das leere Wort enthält, für jedes Alphabet Σ .
- Σ^* ist die Sprache aller Wörter über Σ .
- ε ist nicht allgemein in jeder Sprache L enthalten.

Konkatenation

Die Konkatenation von Sprachen A und B ist definiert als die Menge aller Konkatenationen von Wörtern aus A mit Wörtern aus B :

$$AB = \{uv \mid u \in A, v \in B\}$$

Die Konkatenation von Sprachen ist assoziativ, aber im Allgemeinen nicht kommutativ:

$$(AB)C = A(BC), \quad AB \neq BA$$

Ist A eine Sprache über Σ und B eine Sprache über Γ , so ist AB eine Sprache über $\Sigma \cup \Gamma$.

Kleenesche Hülle

Die Kleenesche Hülle A^* einer Sprache A ist definiert als die Vereinigung aller Potenzen von A :

$$A^* = \bigcup_{k \geq 0} A^k$$

Dabei ist $A^0 = \{\varepsilon\}$ und $A^{k+1} = A^k A$. Die Kleenesche Hülle erfüllt die Eigenschaft:

$$(A^*)^* = A^*$$

Entscheidungsproblem

Sei eine Sprache $L \subseteq \Sigma^*$ gegeben. Das Entscheidungsproblem für L besteht darin, für ein beliebiges Wort $x \in \Sigma^*$ zu entscheiden, ob x in L enthalten ist oder nicht:

$$x \in \Sigma^* \mapsto \begin{cases} \text{JA} & \text{wenn } x \in L, \\ \text{NEIN} & \text{wenn } x \notin L. \end{cases}$$

Reguläre Ausdrücke (RA)

Reguläre Ausdrücke sind ein formale Sprache zur endlichen Beschreibung (möglicherweise unendlicher) Sprachen über einem Alphabet Σ .

Syntax

Die Menge RA_Σ der regulären Ausdrücke über Σ ist induktiv definiert durch:

- $\emptyset, \varepsilon \in \text{RA}_\Sigma$
- $\Sigma \subset \text{RA}_\Sigma$
- $R \in \text{RA}_\Sigma \Rightarrow (R^*) \in \text{RA}_\Sigma$
- $R, S \in \text{RA}_\Sigma \Rightarrow (RS) \in \text{RA}_\Sigma$
- $R, S \in \text{RA}_\Sigma \Rightarrow (R \mid S) \in \text{RA}_\Sigma$

Die Menge RA_Σ der regulären Ausdrücke über dem Alphabet Σ ist eine Sprache über dem Alphabet $\Sigma \cup \{\varepsilon, \emptyset, (,), *, \mid\}$.

Abkürzungen

$$\begin{aligned} R^+ &:= RR^* \\ R? &:= (R \mid \varepsilon) \\ [R_1, \dots, R_k] &:= R_1 \mid \dots \mid R_k \end{aligned}$$

Prioritätsreihenfolge: $() > * > \cdot > \mid$

Semantik

Jedem regulären Ausdruck R wird eine Sprache $L(R)$ zugeordnet:

$$\begin{aligned} L(\emptyset) &= \emptyset \\ L(\varepsilon) &= \{\varepsilon\} \\ L(a) &= \{a\} \quad \text{für } a \in \Sigma \\ L(R^*) &= L(R)^* \\ L(RS) &= L(R)L(S) \\ L(R \mid S) &= L(R) \cup L(S) \end{aligned}$$

Eine Sprache $A \subseteq \Sigma^*$ heisst *regulär*, falls es einen regulären Ausdruck R mit $A = L(R)$ gibt.

Rechenregeln

Für jedes Alphabet Σ und alle regulären Ausdrücke $R, S, T \in \text{RA}_\Sigma$ gelten die folgenden Identitäten:

$$\begin{aligned} L(R \mid S) &= L(S \mid R) \\ L(R \mid (S \mid T)) &= L((R \mid S) \mid T) \\ L(R(S \mid T)) &= L(RS \mid RT) \\ L(R \mid R) &= L(R) \\ L(R(ST)) &= L((RS)T) \\ L((R^*)^*) &= L(R^*) \end{aligned}$$

Endliche Automaten

Grundidee

Endliche Automaten (EA) sind speicherbeschränkte Berechnungsmodelle zur Erkennung formaler Sprachen. Der einzige interne Speicher ist der aktuelle Zustand. Ein Eingabewort wird sequenziell gelesen; Akzeptanz wird über Endzusände entschieden.

Deterministische endliche Automaten

Ein deterministischer endlicher Automat (DEA) ist ein Tupel

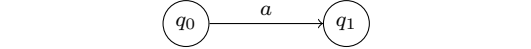
$$M = (Q, \Sigma, \delta, q_0, F)$$

mit

- endlicher Zustandsmenge Q ,
- Alphabet Σ ,
- totaler Übergangsfunktion $\delta : Q \times \Sigma \rightarrow Q$,
- Startzustand $q_0 \in Q$,
- Endzustandsmenge $F \subseteq Q$.

Graphische Darstellung

Der Übergang $\delta(q_0, a) = q_1$ wird grafisch dargestellt als



Konfigurationen

Eine Konfiguration ist ein Paar $(q, w) \in Q \times \Sigma^*$

mit aktuellem Zustand q und Restwort w .

Berechnungsschritte

Die Ein-Schritt-Relation ist definiert durch $(q, w) \vdash_M (p, x)$

genau dann, wenn $w = ax$ mit $a \in \Sigma$ und $\delta(q, a) = p$. Die reflexiv-transitive Hülle wird mit

$$(q, w) \vdash_M^* (p, x)$$

bezeichnet.

Akzeptanz und Sprache

Für $w \in \Sigma^*$ startet die Berechnung in (q_0, w) . w wird akzeptiert, falls

$$(q_0, w) \vdash_M^* (q_f, \varepsilon) \quad \text{für ein } q_f \in F.$$

Die erkannte Sprache ist

$$L(M) = \{w \in \Sigma^* \mid (q_0, w) \vdash_M^* (q_f, \varepsilon) \text{ für ein } q_f \in F\}.$$

Nichtdeterministische endliche Automaten

Ein nichtdeterministischer endlicher Automat (NEA) hat dieselbe Struktur

$$M = (Q, \Sigma, \delta, q_0, F)$$

wobei

$$\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$$

gilt. Damit sind mehrere Folgezustände pro Symbol möglich.

Akzeptanz: Es existiert mindestens ein Lauf, der in einem Zustand aus F endet.

Äquivalenz von DEA und NEA

Ein NEA $N = (Q_N, \Sigma, \delta_N, q_{0N}, F_N)$ kann in einen äquivalenten DEA $D = (Q_D, \Sigma, \delta_D, q_{0D}, F_D)$ überführt werden durch

- $Q_D = \mathcal{P}(Q_N)$ (Mengen von NEA-Zuständen),
- $F_D = \{S \subseteq Q_N \mid S \cap F_N \neq \emptyset\}$ (DEA-Endzustände sind Mengen, die mindestens einen NEA-Endzustand enthalten),
- $\delta_D(S, a) = \bigcup_{q \in S} \delta_N(q, a)$ für $S \subseteq Q_N$ und $a \in \Sigma$ (Menge aller Zustände, die durch a von einem Zustand in S erreichbar sind),

ε -Übergänge

ε -NEAs erlauben Übergänge ohne Symbolkonsum mit

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q).$$

Auch ε -NEAs sind durch Eliminierung von ε -Übergängen und anschließende Determinisierung zu DEA äquivalent.

Äquivalenz DEA und RA

Jede von einem DEA erkannte Sprache kann durch einen regulären Ausdruck beschrieben werden, und umgekehrt. Diese Sprachen werden als reguläre bezeichnet. Reguläre Sprachen sind durch verschiedene Formalismen äquivalent beschreibbar, darunter DEA, NEA, ε -NEA und reguläre Ausdrücke.

- Akzeptierende Mech.:** DEA, NEA, ε -NEA
- Beschreibungsform:** reguläre Ausdrücke

Abschlusseigenschaften regulärer Sprachen

Die Klasse der regulären Sprachen ist unter verschiedenen Operationen abgeschlossen. Seien $L, L_1, L_2 \subseteq \Sigma^*$ reguläre Sprachen. Dann sind ebenfalls regulär:

- Vereinigung: $L_1 \cup L_2$
- Konkatenation: $L_1 L_2$
- Kleenesche Hülle: L^*
- Komplement: $\overline{L} = \Sigma^* \setminus L$
- Schnitt: $L_1 \cap L_2$
- Mengendifferenz: $L_1 \setminus L_2$

Zustandsklassen

Zu jedem Zustand p eines Automaten M gehört die Zustandsklasse

$$[p] = \{w \in \Sigma^* \mid M \text{ endet nach dem Lesen von } w \text{ in } p\}.$$

Die Zustandsklassen besitzen folgende Eigenschaften:

- Die Menge aller Wörter wird durch die Klassen partitioniert:

$$\Sigma^* = \bigcup_{p \in Q} [p]$$

- Für verschiedene Zustände sind die Klassen disjunkt:

$$[p] \cap [q] = \emptyset \quad \text{für } p \neq q$$

- Die von M akzeptierte Sprache ergibt sich als Vereinigung der Klassen akzeptierender Zustände:

$$L(M) = \bigcup_{p \in F} [p]$$

Grenzen endlicher Automaten

Ein endlicher Automat hat nur endlich viele Zustände und deshalb nur begrenzten Speicher. Landen zwei Wörter x und y im selben Zustand, kann der Automat sie nicht mehr unterscheiden. Daher gilt für jedes Fortsetzungswort $z \in \Sigma^*$:

$$xz \in L(M) \iff yz \in L(M).$$

Wörter derselben Zustandsklasse verhalten sich also bei jeder Fortsetzung gleich. Damit kann man Mindestzahlen für die benötigten Zustände eines Automaten ableiten.

Kontextfreie Grammatiken

Eine kontextfreie Grammatik (KFG) ist ein Tupel

$$G = (N, \Sigma, P, S)$$

mit

- Alphabet der **Nichtterminalen** N ,
- Alphabet der **Terminalen** Σ mit $N \cap \Sigma = \emptyset$,
- Endliche Menge **Produktionen** $P \subseteq N \times (N \cup \Sigma)^*$,
- Startsymbol** $S \in N$.

Eine Produktion hat die Form

$$A \rightarrow \alpha \quad \text{mit } A \in N, \alpha \in (N \cup \Sigma)^*.$$

$\alpha \in (N \cup \Sigma)^*$ wird auch als **Satzformel** bezeichnet.

Ableitung

Der Ableitungsschritt ist definiert durch

$$uAv \Rightarrow_G u\alpha v$$

genau dann, wenn $A \rightarrow \alpha \in P$ und $u, v \in (N \cup \Sigma)^*$. Die reflexiv-transitive Hülle wird mit $\Rightarrow_G^*$ bezeichnet.

Erzeugte Sprache

Die von G erzeugte Sprache ist

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow_G^* w\}.$$

Eine Sprache $L \subseteq \Sigma^*$ heisst kontextfrei, falls es eine KFG G mit $L = L(G)$ gibt.

Links- und Rechtsableitung

- Linksableitung:** In jedem Schritt wird das linkeste Nichtterminal ersetzt.
- Rechtsableitung:** In jedem Schritt wird das rechteste Nichtterminal ersetzt.

Beide beschreiben dieselbe erzeugte Sprache.

Ableitungsbaum

Ein Ableitungsbaum repräsentiert die hierarchische Struktur einer Ableitung:

- Wurzel ist S .
- Innere Knoten sind Nichtterminale.
- Blätter sind Terminalsymbole.

Die von links nach rechts gelesenen Blätter ergeben das erzeugte Wort.

Mehrdeutigkeit

Eine Grammatik ist mehrdeutig, wenn es ein Wort $w \in L(G)$ mit mindestens zwei verschiedenen Ableitungsbäumen gibt. Eine Sprache heisst inhärent mehrdeutig, wenn jede Grammatik, die sie erzeugt, mehrdeutig ist.

Kellerautomaten

Kellerautomaten (KA) erweitern endliche Automaten um einen Stack (Keller) als Speicherstruktur mit LIFO-Zugriff. Sie erkennen genau die Klasse der kontextfreien Sprachen.

Deterministischer Kellerautomat

Ein deterministischer Kellerautomat ist ein Tupel

$$M = (Q, \Sigma, \Gamma, \delta, q_0, \$, F)$$

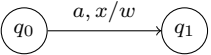
wobei

- Q eine endliche Menge von Zuständen,
- Σ ein endliches Eingabealphabet,
- Γ ein endliches Kelleralphabet,
- $\delta : Q \times \Sigma \times \Gamma \rightarrow Q \times \Gamma^*$ eine Übergangsfunktion,
- $q_0 \in Q$ der Startzustand,
- $\$ \in \Gamma$ das Startkellersymbol,
- $F \subseteq Q$ die Menge der akzeptierenden Zustände.

Damit der Determinismus gewährleistet ist, gilt für jeden Zustand q und alle Symbole $a \in \Sigma$ und $x \in \Gamma$, wenn $\delta(q, a, x)$ definiert ist, dann ist $\delta(q, \varepsilon, x)$ nicht definiert und umgekehrt.

Graphische Darstellung

Der Übergang $\delta(q_0, a, x) = (q_1, w)$ wird grafisch dargestellt als



Nichtdeterministischer Kellerautomat

Ein nichtdeterministischer Kellerautomat wird durch den selben Tupel wie der deterministische definiert, jedoch ist die Übergangsfunktion:

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma^*)$$

Sie sind mächtiger als deterministische Kellerautomaten. Bsp: $\{aa^{\mathcal{R}} \mid a \in \{0, 1\}^*\}$

Konfigurationen

Eine Konfiguration ist ein Tripel

$$(q, w, \gamma) \in Q \times \Sigma^* \times \Gamma^*$$

mit aktuellem Zustand q , Restwort w und Kellerinhalt γ (oberstes Symbol).

- $(q_0, w, \$)$ ist die Startkonfiguration.
- (q, ε, γ) ist eine Endkonfiguration.

Berechnungsschritt

Ein Schritt

$$(q, aw, x\beta) \vdash_M (p, w, \gamma\beta)$$

ist möglich, falls $(p, \gamma) \in \delta(q, a, x)$ für $a \in \Sigma$. Ein ε -Schritt

$$(q, w, x\beta) \vdash_M (p, w, \gamma\beta)$$

ist möglich, falls $(p, \gamma) \in \delta(q, \varepsilon, x)$. Die reflexiv-transitive Hülle wird mit $\vdash_M^*$ bezeichnet.

Akzeptanz

Ein Wort w wird von M akzeptiert, wenn es eine Berechnung gibt, die von der Startkonfiguration $(q_0, w, \$)$ zu einer Endkonfiguration $(q_f, \varepsilon, \gamma)$ mit $q_f \in F$ und $\gamma \in \Gamma^*$ führt:

$$w \in L(M) \iff (q_0, w, \$) \vdash_M^* (q_f, \varepsilon, \gamma)$$

Beziehung zu Grammatiken

Eine Sprache ist kontextfrei genau dann, wenn es einen *nichtdeterministischen* Kellerautomaten gibt, der sie akzeptiert.

Turingmaschinen

Grundidee

Eine Turing-Maschine (TM) ist ein endlicher Automat, ergänzt um einen Lese-/Schreibkopf und ein beidseitig unendliches Band von Zellen. Die Eingabe steht zu Beginn auf dem Band; alle übrigen Zellen enthalten das Leerzeichen $\sqcup$.

Deterministische Turing-Maschine

Eine deterministische TM ist ein 7-Tupel

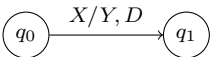
$$M = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, F)$$

mit

- endlicher Zustandsmenge Q ,
- Eingabealphabet Σ mit $\Sigma \subset \Gamma$,
- Bandalphabet Γ (endliche Menge von Symbolen),
- Übergangsfunktion $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$,
- Startzustand $q_0 \in Q$,
- Leerzeichen $\sqcup \in \Gamma \setminus \Sigma$,
- Menge akzeptierender Zustände $F \subseteq Q$.

Graphische Darstellung

Der Übergang $\delta(q_0, X) = (q_1, Y, D)$ wird dargestellt als



wobei $D \in \{L, R\}$ die Bewegungsrichtung des Kopfes angibt.

Konfiguration

Eine Zeichenreihe

$$K = X_1 X_2 \dots X_{i-1} q X_i X_{i+1} \dots X_n$$

heisst Konfiguration von M , wobei $q \in Q$ der aktuelle Zustand ist und der Kopf über X_i steht.

Startkonfiguration: $K_0 = q_0 X_1 X_2 \dots X_n$ (Kopf auf erstem Zeichen).

Berechnungsschritte

Für $\delta(q, X_i) = (p, Y, R)$ (Bewegung nach rechts):

$$\dots X_{i-1} q X_i \dots \vdash \dots X_{i-1} Y p X_{i+1} \dots$$

Für $\delta(q, X_i) = (p, Y, L)$ (Bewegung nach links):

$$\dots X_{i-1} q X_i \dots \vdash \dots X_{i-2} p X_{i-1} Y X_{i+1} \dots$$

Berechnung

Eine Berechnung auf Eingabe $w \in \Sigma^*$ ist eine endliche Folge

$$K_0 \vdash K_1 \vdash \dots \vdash K_n$$

wobei K_0 die Startkonfiguration ist und aus K_n keine Bewegung mehr möglich ist. Kurzschreibweise: $K_0 \vdash^* K_n$.

Sprache einer TM

Die von M akzeptierte Sprache ist

$$L(M) = \{w \mid q_0 w \vdash^* \alpha p \beta, p \in F \text{ und } \alpha, \beta \in \Gamma^*\}.$$

Eine Sprache, die von einer TM akzeptiert wird, heisst **rekursiv aufzählbar**. Die Klasse der rekursiv aufzählbaren Sprachen umfasst u. a. alle kontextfreien Sprachen.

Erweiterungen von TM

TM mit Speichern

In der Zustandssteuerung werden zusätzlich endlich viele Datenzustände gespeichert. Ist nur eine Vereinfachung: $Q_{\text{neu}} = Q \times \Gamma^k$ für k Speicher.

TM mit mehreren Spuren

Das Band hat n Spuren; jede Zelle speichert ein n -Tupel. Simulation durch $\Gamma_{\text{neu}} = \Gamma^n$.

TM mit mehreren Bändern

Die TM besitzt k Bänder mit je einem unabhängigen Lese-/Schreibkopf. Übergangsfunktion:

$$\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{R, L, S\}^k$$

Satz: Jede Sprache, die von einer mehrbändigen TM akzeptiert wird, wird auch von einer TM mit nur einem Band akzeptiert (Simulation mit $2k$ Spuren).

Nichtdeterministische TM (NTM)

Eine NTM hat die Übergangsfunktion

$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times D).$$

Satz: Jede Sprache, die von einer NTM akzeptiert wird, wird auch von einer deterministischen TM akzeptiert (Simulation über Breitensuche aller Konfigurationen).

Beschränkungen von TM

TM mit semi-unendlichem Band

Das Band ist nur in eine Richtung unendlich.

Satz: Jede TM kann durch eine TM mit semi-unendlichem Band und zwei Spuren simuliert werden.

Maschine mit mehreren Stacks

Eine k -Stack-Maschine ist ein DKA mit k Stacks.

$$\delta : Q \times \Gamma_1 \times \dots \times \Gamma_k \rightarrow Q \times \Gamma_1^* \times \dots \times \Gamma_k^*$$

Satz: Jede TM kann durch eine 2-Stack-Maschine simuliert werden (Bewegung durch Verschieben der Inhalte zwischen den beiden Stacks).

Zählermaschine

Eine Zählermaschine (Counter Machine) mit k Zählern entspricht einer k -Stack-Maschine, bei der Stacks durch Zähler in $\mathbb{N}$ ersetzt werden. Nur $= 0 / \neq 0$ unterscheidbar; Inkrement/Dekrement um 1. **Satz:** Jede TM kann durch eine 2-Zähler-Maschine simuliert werden. (via: TM $\rightarrow$ 2-Stack $\rightarrow$ 3-Zähler $\rightarrow$ 2-Zähler).

Stack als Zahl (Basis k): Der Zustand $s = X_1 X_2 \dots X_n$ eines Stacks mit $k - 1$ Stacksymbolen kann durch einen Zähler z kodiert werden.

$$z = \sum_{i=0}^{n-1} X_{n-i} \cdot k^{n-i-1}$$

$$\text{pop} : z_{\text{new}} = z \div k, \quad X = z \bmod k$$

$$\text{push}(X_i) : z_{\text{new}} = z \cdot k + X_i$$

Der Dritte Zähler k dient zur Berechnung.

3 Zähler $\rightarrow$ 2 Zähler: Die drei Zähler mit den Werten i, j, k werden als $z = 2^i \cdot 3^j \cdot 5^k$ in einem Zähler kodiert. Mit dem zweiten Zähler wird gerechnet.

Universelle Turingmaschine

Eine **universelle TM** (UTM) simuliert jede beliebige TM M , sofern deren Codierung Cod_M zusammen mit der Eingabe w übergeben wird.

Kodierung einer TM

Jede TM lässt sich als Binärzeichenreihe über $\{0, 1\}$ kodieren:

- Zustände:** $q_1 = \text{start}, q_2 = \text{akzeptierend}, q_3, \dots, q_n = \text{weitere Zustände}.$
- Bandsymbole:** $X_1 \hat{=} 0, X_2 \hat{=} 1, X_3 \hat{=} \sqcup, X_4, \dots, X_n = \text{weitere Symbole}.$
- Richtungen:** $D_1 \hat{=} L, D_2 \hat{=} R.$
- Transition:** $\delta(q_i, X_j) = (q_k, X_l, D_m)$ wird kodiert als
$$0^i 1 0^j 1 0^k 1 0^l 1 0^m$$
- Gesamte TM:** Alle Transitionen C_i werden durch 11 getrennt:
$$C_1 11 C_2 11 C_3 11 \dots$$

Durch Voranstellen einer 1 entsteht eine eindeutige Zahl (Gödelnummer).

Definition

Eine TM M_1 heisst **universelle Turingmaschine**, wenn sie die Eingabe $\text{Cod}_M 111w$ verarbeitet und das Ergebnis von M auf w liefert (für alle M, w). Die Folge 111 trennt die TM-Kodierung von der Eingabe w .

Berechenbarkeitstheorie

Church-Turing-These

Intuitiv berechenbar: Funktionen, die durch ein mechanisches Verfahren berechnet werden können.

Turing-berechenbar: Funktionen, die von einer TM berechnet werden können.

Church-Turing-These: Die Klasse der Turing-berechenbaren Funktionen stimmt mit der Klasse der intuitiv berechenbaren Funktionen überein.

Gandy's These M: Alles, was mit einer endlichen Maschine berechnet werden kann, ist bereits Turing-berechenbar. *Nicht formal beweisbar.*

Turing-berechenbare Funktionen

Eine TM $T = (Q, \Sigma, \Gamma, \dots)$ als partielle Funktion $T : \Sigma^* \rightarrow \Gamma^*$:

$$T(w) = \begin{cases} u & \text{falls } T \text{ auf } w \text{ mit } u \text{ auf dem Band anhält} \\ \uparrow & \text{falls } T \text{ bei Input } w \text{ nicht anhält} \end{cases}$$

$f : \Sigma^* \rightarrow \Gamma^*$ heisst **Turing-berechenbar**, wenn es eine TM T gibt mit $T(w) = f(w)$ für alle w wo f definiert ist.

LOOP-Programme

Syntax:

- Zuweisungen:** $x = y + c$ und $x = y - c$ wobei x, y Variablen und $c \in \mathbb{N}$ ist.

- Sequenzen:** P ; Q

- Schleifen:** Loop x Do P End

Semantik: $P_k(n_1, \dots, n_k) =$ Wert von x_0 nach Ablauf, mit n_i in x_i .

Konventionen: x_0 und Hilfsvariablen mit 0 initialisiert. Subtraktion ≥ 0 .

Bedingte Ausführung (IF x1=0 THEN P)

```
x2 = x2 + 1;
Loop x1 Do x2 = x2 - 1 End;
Loop x2 Do P End
```

Verzweigung (IF x1=0 THEN P ELSE Q)

```
x2 = x2 + 1; x3 = x3 + 1;
Loop x1 Do x2 = x2 - 1; x4 = x3 + 0 End;
Loop x2 Do P End;
Loop x4 Do Q End
```

WHILE-Programme

Erweitern LOOP um: While x > 0 Do P End

- Jedes LOOP-Programm ist ein WHILE-Programm
- WHILE-Programme terminieren **nicht** immer

GOTO-Programme

Syntax:

- Mk:** xi = xj + c / Mk: xi = xj - c

- Mk:** If xi = c Then Goto Mr

- Mk:** Goto Mr

- Mk:** HALT

- Sequenzen: P ; Q (keine gemeinsamen Marker)

Primitiv rekursive Funktionen

Grundfunktionen

- Konstante:** $c_k^n : \mathbb{N}^n \rightarrow \mathbb{N}, c_k^n(x_1, \dots, x_n) = k$

- Nachfolger:** $\eta : \mathbb{N} \rightarrow \mathbb{N}, \eta(x) = x + 1$

- Projektion:** $\pi_k^n : \mathbb{N}^n \rightarrow \mathbb{N}, \pi_k^n(x_1, \dots, x_n) = x_k$

Abschluss durch Einsetzung

Sind $f, g_1, \dots, g_k$ primitiv rekursiv mit $g_i : \mathbb{N}^n \rightarrow \mathbb{N}, f : \mathbb{N}^k \rightarrow \mathbb{N}$, dann auch:

$$h(\vec{x}) = f(g_1(\vec{x}), \dots, g_k(\vec{x}))$$

Abschluss durch primitive Rekursion

Sind $h : \mathbb{N}^n \rightarrow \mathbb{N}$ und $g : \mathbb{N}^{n+2} \rightarrow \mathbb{N}$ primitiv rekursiv, dann auch $f : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ mit:

$$f(0, \vec{x}) = h(\vec{x})$$

$$f(k + 1, \vec{x}) = g(f(k, \vec{x}), k, \vec{x})$$

Beispiele

Addition: $\text{Add}(0, y) = \pi_1^1(y),$
 $\text{Add}(x + 1, y) = \eta(\pi_2^3(\text{Add}(x, y), x, y))$
Multiplikation: $\text{Mult}(0, y) = c_0^1(y),$
 $\text{Mult}(x + 1, y) = \text{Add}(\text{Mult}(x, y), \pi_2^2(x, y))$

Unvollständigkeit von LOOP / Prim. Rek.

LOOP-Programme und primitiv rekursive Funktionen sind **nicht** Turing-vollständig. Es gibt totale, berechenbare Funktionen, die nicht LOOP-berechenbar sind:

Ackermannfunktion

$$a(0, m) = m + 1$$

$$a(n + 1, 0) = a(n, 1)$$

$$a(n + 1, m + 1) = a(n, a(n + 1, m))$$

Eigenschaften:

- Turing-berechenbar (WHILE-Programm möglich)
- Nicht** primitiv rekursiv / LOOP-berechenbar
- Total (überall definiert) – Beweis via Induktion über lexikograph. Ordnung auf $\mathbb{N} \times \mathbb{N}$
- Wächst schneller als jede primitiv rekursive Funktion

LOOP-Interpreter

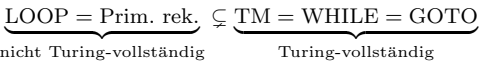
$I : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit $I(\langle P \rangle, x) = P_1(x)$ für jedes LOOP-Programm P .

- Satz:**
- I ist total und eindeutig (LOOP terminiert immer)
 - I ist Turing-berechenbar
 - I ist **nicht** LOOP-berechenbar

Beweis (Widerspruch): Angenommen P_2 ist LOOP-Interpreter. Definiere LOOP-Programm Q mit $Q_1(x) = P_2(x, x) + 1$. Dann:

$$Q_1(\langle Q \rangle) = P_2(\langle Q \rangle, \langle Q \rangle) + 1 = Q_1(\langle Q \rangle) + 1 \quad \text{!}$$

Überblick Berechnungsmodelle



Entscheidbarkeit

Entscheidbarkeit

Eine Sprache $A \subset \Sigma^*$ heisst **entscheidbar**, wenn eine TM T existiert, die das Entscheidungsproblem (Σ, A) löst:

- $x \in A \Rightarrow T$ hält mit Bandinhalt “1” (Ja)
- $x \in \Sigma^* \setminus A \Rightarrow T$ hält mit Bandinhalt “0” (Nein)

T muss bei **jeder** Eingabe $x \in \Sigma^*$ nach endlich vielen Schritten halten.

Äquivalent: A ist entscheidbar $\Leftrightarrow (\Sigma, A)$ mit einem WHILE-Programm lösbar (Entscheidungsverfahren).

Semi-Entscheidbarkeit

$A \subset \Sigma^*$ heisst **semi-entscheidbar**, wenn eine TM T existiert mit:

- $x \in A \Rightarrow T$ hält mit Bandinhalt “1” (Ja)
- $x \in \Sigma^* \setminus A \Rightarrow T$ hält **nie** an

Liefert nur positive Antworten; statt “Nein” keine Antwort.

Konvention (natürliche Zahlen)

$X \subset \mathbb{N}$ ist (semi-) entscheidbar $\Leftrightarrow \{bin(x) \mid x \in X\} \subset \{0, 1\}^*$ ist (semi-) entscheidbar.

Zusammenhänge

- Jede entscheidbare Sprache ist auch semi-entscheidbar.
- A ist entscheidbar $\Leftrightarrow$ sowohl A als auch $\bar{A}$ sind semi-entscheidbar.

Beweis ($\Leftarrow$): Simuliere semi-Entscheidungsverfahren für A und $\bar{A}$ schrittweise parallel:

```
INPUT(w); n = 0;
WHILE true DO
  n = n + 1;
  IF A(w,n) THEN return 1;
  IF B(w,n) THEN return 0
END
```

$A(w, n)$: semi-EV von A terminiert nach n Schritten;
 $B(w, n)$: analog für $\bar{A}$.

Abschlusseigenschaften

- A entscheidbar $\Leftrightarrow \bar{A}$ entscheidbar
- A, B (semi-) entscheidbar $\Rightarrow A \cup B, A \cap B$ (semi-) entscheidbar

Charakterisierungen (Semi-Entscheidbarkeit)

Für $A \subset \Sigma^*$ sind äquivalent:

- A ist rekursiv aufzählbar
- A ist semi-entscheidbar
- A ist Wertebereich einer totalen berechenbaren Funktion
- A ist Definitionsbereich einer berechenbaren Funktion

Reduktion

$A \subset \Sigma^*$ heisst auf $B \subset \Gamma^*$ **reduzierbar** ($A \preceq B$), wenn es eine totale, Turing-berechenbare Funktion $F : \Sigma^* \rightarrow \Gamma^*$ gibt mit:

$$w \in A \iff F(w) \in B$$

Transitivität: $A \preceq B$ und $B \preceq C \Rightarrow A \preceq C$
Satz: Ist B (semi-) entscheidbar und $A \preceq B$, dann ist auch A (semi-) entscheidbar.

Kontraposition: Ist A unentscheidbar und $A \preceq B$, dann ist auch B unentscheidbar.

Beweis (Entscheidbarkeit):

```
INPUT(w); u = F(w); RETURN P(u)
```

Halteprobleme

Spezielles Halteproblem

$H_S := \{w \in \{0, 1\}^* \mid T_w \text{ angesetzt auf } w \text{ hält}\}$
Gegeben: Code w einer TM T_w .
Gefragt: Hält T_w auf eigenem Code w ?

Allgemeines Halteproblem

$H := \{w\#x \in \{0, 1, \#\}^* \mid T_w \text{ angesetzt auf } x \text{ hält}\}$
Gegeben: Code w einer TM T_w und Input x .
Gefragt: Hält T_w auf Eingabewort x ?

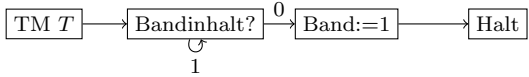
Leeres Halteproblem

$H_0 := \{w \in \{0, 1\}^* \mid T_w \text{ angesetzt auf } \varepsilon \text{ Band hält}\}$
Gegeben: Code w einer TM T_w .
Gefragt: Hält T_w auf leerem Band?

Unentscheidbarkeit der Halteprobleme

Unentscheidbarkeit von H_S

Annahme: TM T entscheidet H_S . Konstruiere TM P :



Sei w der Code von P ($T_w = P$):

- P auf w hält $\Leftrightarrow T(w) = 0 \Leftrightarrow T_w$ auf w hält nicht $\Leftrightarrow P$ auf w hält nicht !

Unentscheidbarkeit von H

Reduktion $H_S \preceq H$ via $F(x) = x\#x$.
 $w \in H_S \Leftrightarrow T_w$ hält auf $w \Leftrightarrow w\#w \in H \Leftrightarrow F(w) \in H$

Unentscheidbarkeit von H_0

Reduktion $H \preceq H_0$: Für Input $w\#x$ konstruiere TM T' , die zuerst x auf das leere Band schreibt und sich dann wie T_w verhält.
 T_w hält auf $x \Leftrightarrow T'$ hält auf leerem Band.

Semi-Entscheidbarkeit der Halteprobleme

H_0, H_S und H sind semi-entscheidbar.
Wegen $H_S \preceq H \preceq H_0$ genügt es, H_0 als semi-entscheidbar nachzuweisen: Universelle TM U simuliert T_w auf leerem Band; falls U hält $\rightarrow$ Band := 1.

Satz von Rice

Ist R die Menge aller berechenbaren Funktionen und $S \subset R$ eine **echte, nichtleere** Teilmenge, dann ist

$$C(S) = \{w \in \{0, 1\}^* \mid F_w \in S\}$$

unentscheidbar.
Konsequenzen: Es ist im Allgemeinen unmöglich mechanisch zu prüfen, ob ein Programm...

- eine bestimmte Spezifikation erfüllt
- frei von Bugs ist
- bei jeder Eingabe terminiert
- dieselbe Funktionalität wie ein anderes Programm hat

Komplexitätstheorie

Zeitkomplexität (TM)

$\text{Time}_M(w)$ = Anzahl Konfigurationsübergänge von M auf w .

Worst-Case:
 $\text{Time}_M(n) = \max\{\text{Time}_M(w) \mid |w| = n\}$

O-Notation (Landau-Symbole)

- $f \in \mathcal{O}(g)$: $\exists n_0, c: \forall n \geq n_0: f(n) \leq c \cdot g(n)$ (obere Schranke)
- $f \in \Omega(g)$: $\exists n_0, d: \forall n \geq n_0: f(n) \geq \frac{1}{d} \cdot g(n)$ (untere Schranke)
- $f \in \Theta(g)$: $f \in \mathcal{O}(g) \cap \Omega(g)$ (asymptotisch gleich)

Regeln: $c \cdot f(n) \in \mathcal{O}(f(n))$; $c \in \mathcal{O}(1)$;
 $a_k n^k + \dots + a_0 \in \mathcal{O}(n^k)$; transitiv.
Abfolge: $1 \ll \log n \ll n \ll n \log n \ll n^2 \ll 2^n \ll n!$

Komplexitätsklassen

P: Alle Probleme, die eine **det. TM** in Polynomzeit löst.
NP: Alle Probleme, die eine **NTM** in Polynomzeit löst. Äquivalent dazu: Menge aller Sprachen mit Polynomzeit-Verifizierer.
P $\hat{=}$ Lösung *finden* in Polyzeit; **NP** $\hat{=}$ Lösung *verifizieren* in Polyzeit.
Offene Frage: **P** = **NP**? Vermutung: **P** $\neq$ **NP**.

NP-schwer: L ist NP-schwer, falls $\forall L' \in \mathbf{NP}: L' \preceq_p L$. Es existiert also eine polynomielle Reduktion von jedem NP-Problem auf L .
NP-vollständig: $L \in \mathbf{NP}$ und L ist NP-schwer. Falls *ein* NP-vollständiges $L \in \mathbf{P} \Rightarrow \mathbf{P} = \mathbf{NP}$.

Polynomzeit-Verifizierer

Ein Problem $A \in NP$ falls ein Verifizierungsproblem $A' \in P$ existiert, so dass

$$w \in A \Leftrightarrow \exists z : (|z| \leq p(|w|) \wedge z\#w \in A')$$

Polynomielle Reduktion

$A \preceq_p B$: Es existiert eine in Polynomzeit berechenbare Funktion f mit $w \in A \Leftrightarrow f(w) \in B$.
Bedeutet: B ist mind. so schwer wie A .
Satz: P_1 NP-schwer, $P_2 \in \mathbf{NP}$, $P_1 \preceq_p P_2 \Rightarrow P_2$ NP-vollständig.

SAT und Satz von Cook

SAT: Ist eine KNF-Formel erfüllbar?
Satz von Cook: SAT ist NP-vollständig.
 $SAT \in NP$: Zeuge = Belegung; Verifikation in Polyzeit.
NP-schwer: Berechnung einer polyzeit-NTM als KNF kodierbar.

CLIQUE ist NP-vollständig

Clique: $S \subseteq V$ paarweise verbunden.
Problem: $\exists$ Clique der Grösse k ?
 $CLIQUE \in NP$: Zeuge = S ; Verifikation in $\mathcal{O}(n^2)$.
 $SAT \preceq_p CLIQUE$: Aus KNF φ mit m Klauseln: $k = m$, Knoten $[i, j]$ pro Literal, Kanten zwischen versch. Klauseln falls Literale nicht komplementär.
 φ erfüllbar $\Leftrightarrow G$ hat Clique der Grösse k .

Chomsky-Hierarchie

